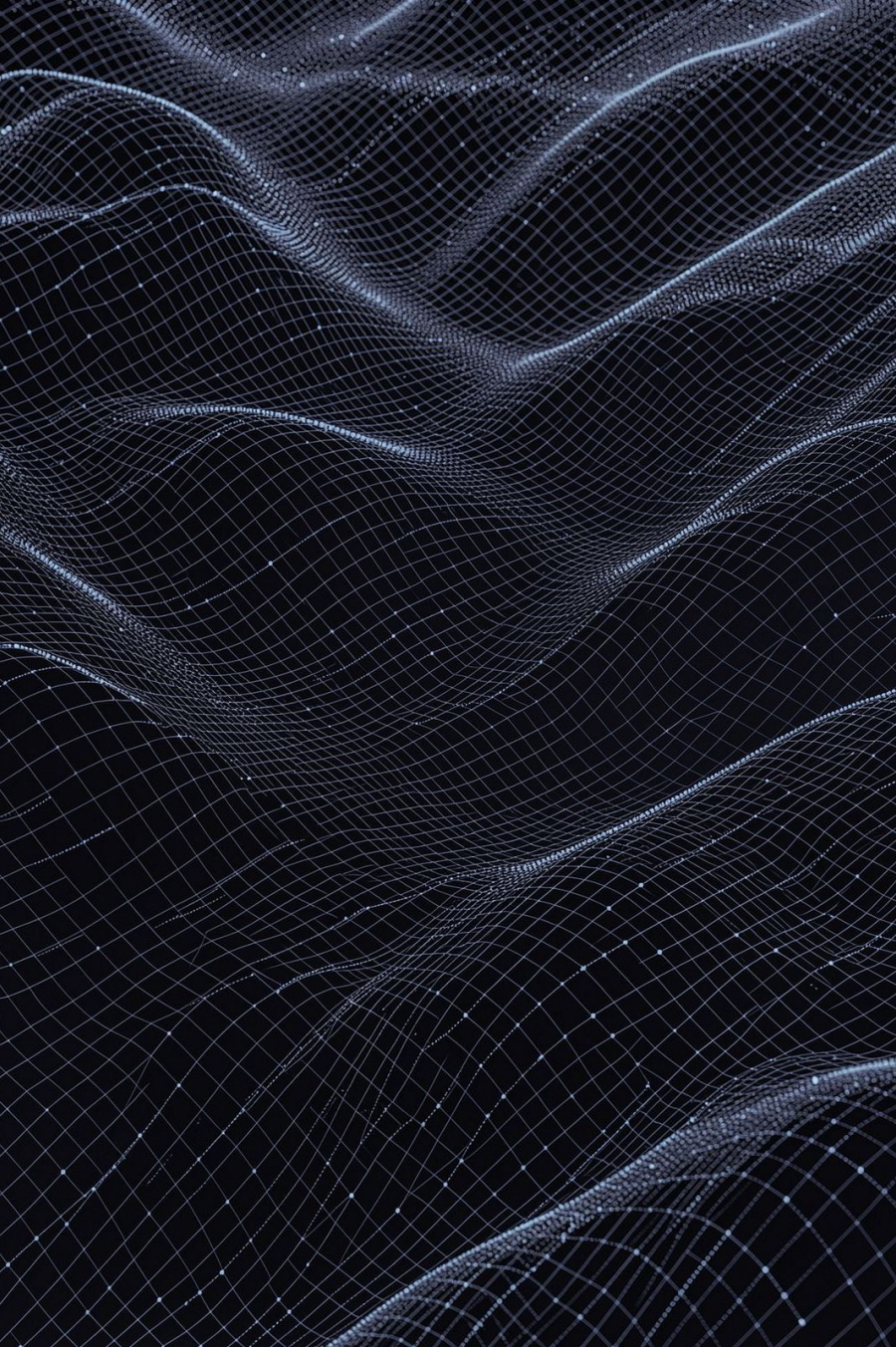


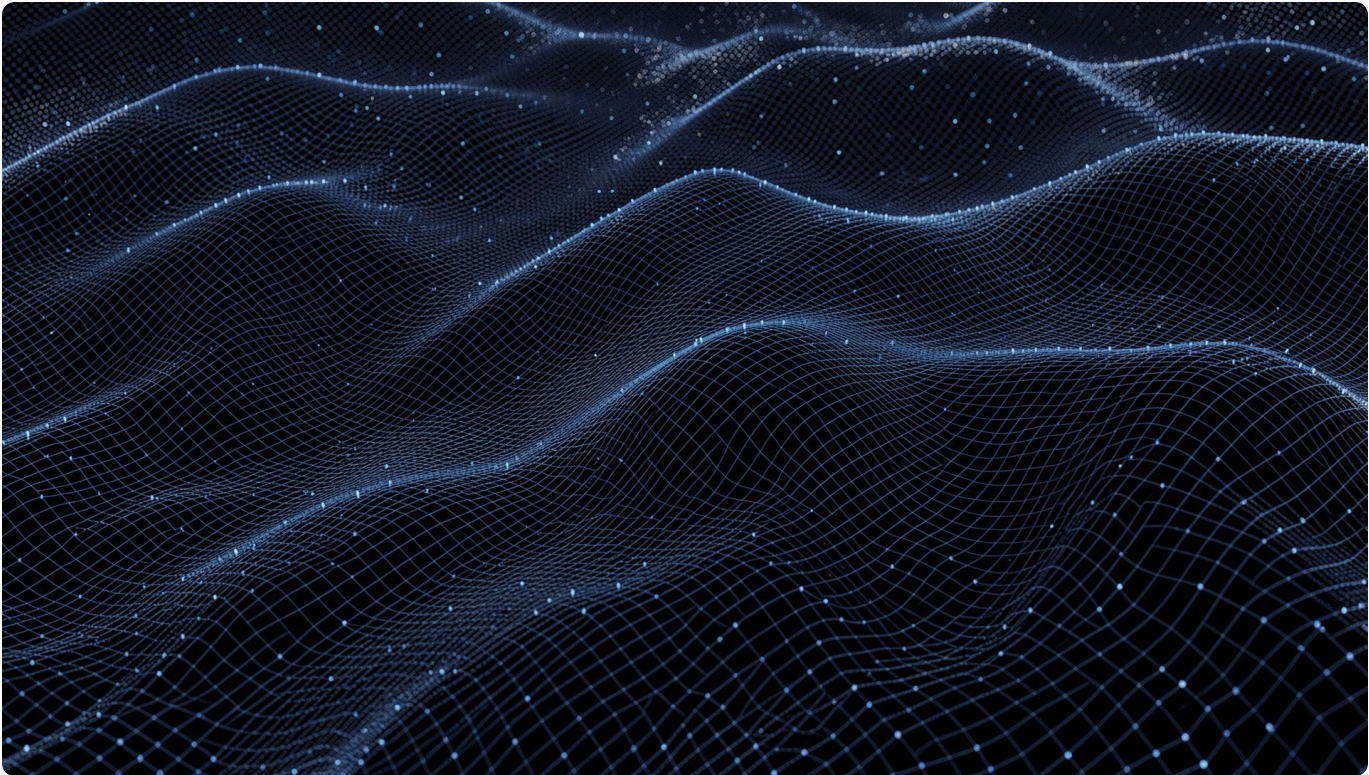


Introduction to Agentic AI, LLM as Reasoning Engine & Tool-Using Agents

Dr. Anilkumar V. Brahmane

Computer Engineering Department · Sanjivani College of Engineering

What is Traditional AI vs. Agentic AI?



Traditional AI

Follows fixed rules or scripts — like a vending machine, it only does what it's programmed to do, step by step.

Agentic AI

Given a **goal**, it decides its own steps, picks tools, adapts when something goes wrong, and keeps going until done.

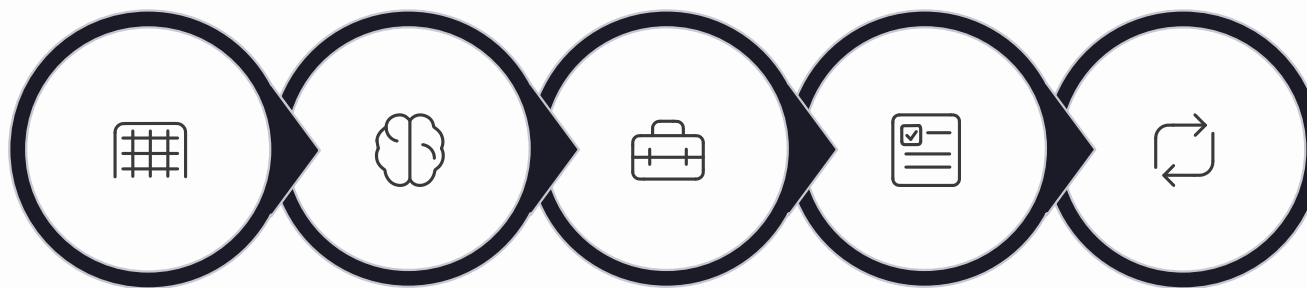
Key Difference

Autonomy — an agent chooses its own steps and acts; a chatbot only answers questions.

Property	Chatbot	Scripted Automation	AI Agent
Responds to	One message	Fixed steps	A goal
Can use tools?	No	Hard-coded only	Yes, dynamically
Adapts to failure?	No	No	Yes
Example	ChatGPT Q&A	Excel macro	Auto-booking agent

What is Agentic AI? — Simple Definition

✈️ Think of asking a friend to "book the cheapest flight to Mumbai next week." They search flights, compare prices, check dates, and book — all on their own. **That's an agent!**



User Goal

Agent Brain

Select Tools

Get Result

Loop Back

The Agent Formula

Agent = LLM + Goal + Tools + Loop + Memory



Not just a chatbot

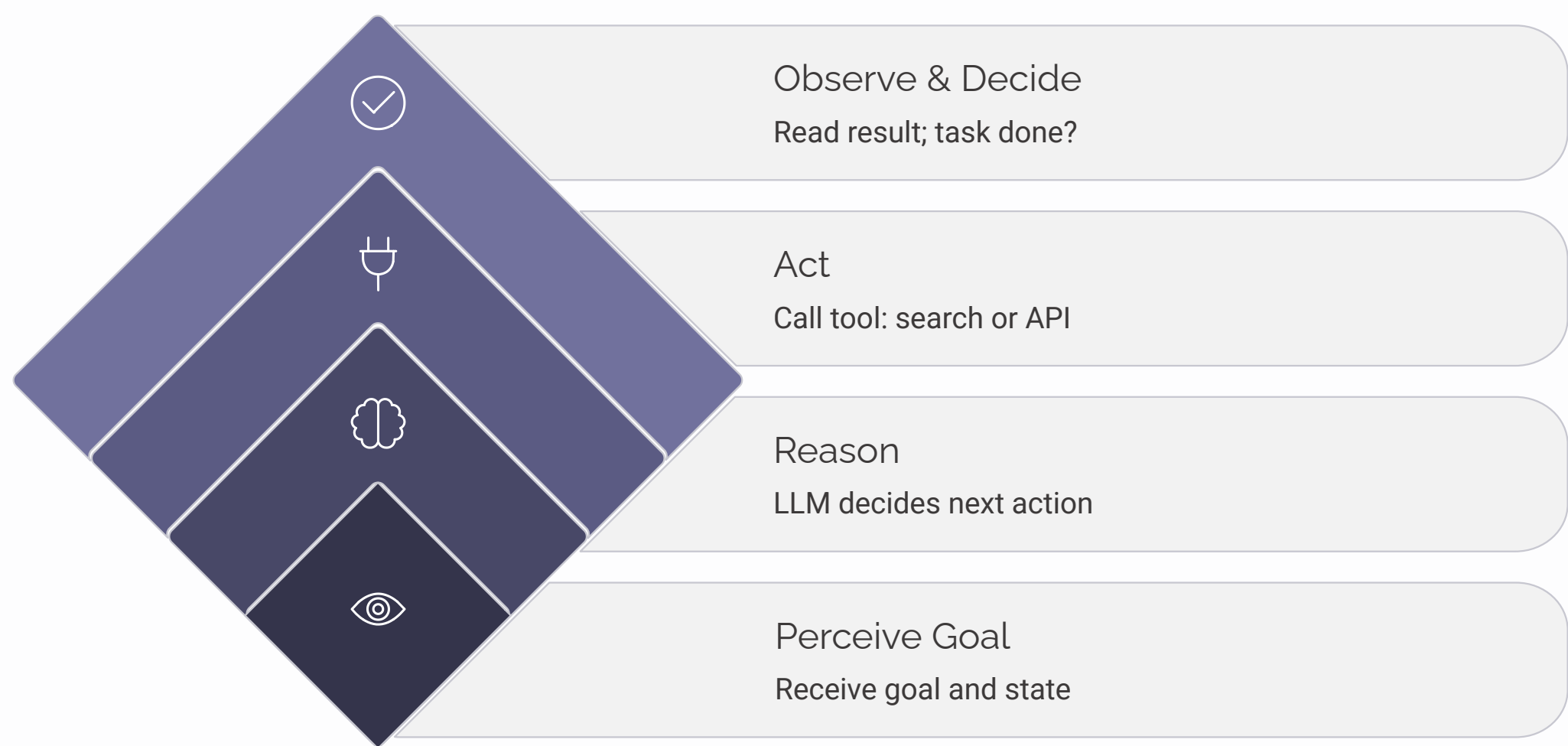
A chatbot *tells you* how to reset a password; an agent *actually resets it* and emails the user.



Goal-driven

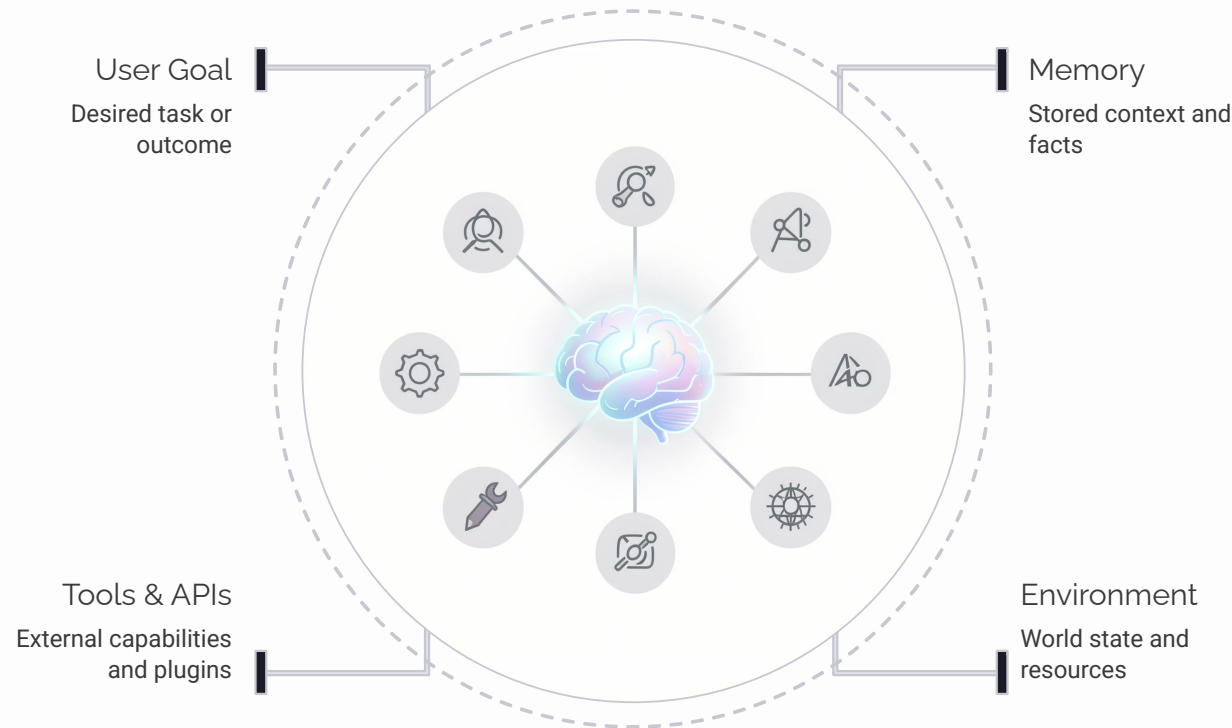
The agent keeps looping — reasoning, acting, observing — until the goal is fully achieved.

The Agent Loop — How It Works Step by Step



- 1 · Perceive**
Receives the goal and current state
- 2 · Reason**
LLM thinks: "What should I do next?"
- 3 · Act**
Calls a tool — search, calculator, or API
- 4 · Observe**
Reads the tool's result
- 5 · Repeat**
Returns to Reason until task is complete

LLM as the Reasoning Engine — The Brain of the Agent



What is an LLM?

A Large Language Model (e.g., GPT-4, Gemini) trained on billions of documents — predicts the best next word or action.

Plain LLM Limitation

Text in → Text out. It has **no hands** — cannot search the web, run code, or check live data.

As Reasoning Engine

The LLM reads the goal + tool descriptions, then **decides which tool to call and why** — it plans every step like a brain.

Chain-of-Thought — How LLMs Reason Step by Step

Analogy

Like a student showing their work in a math exam — instead of jumping to the answer, the LLM writes out its thinking step by step.

Example

"What is 15% of 240?"

→ LLM thinks:

$$15\% = 15/100$$

$$15 \times 240 = 3600$$

$$3600 \div 100 = 36$$

Answer: 36

Chain-of-Thought (CoT)

LLM breaks a complex problem into smaller sub-steps **before** giving the final answer.

- ❏ CoT is automatically used by modern agents — it's what makes LLMs reliable reasoning engines, not just text completers.

Reduces Errors

Stepwise thinking catches mistakes early

Improves Accuracy

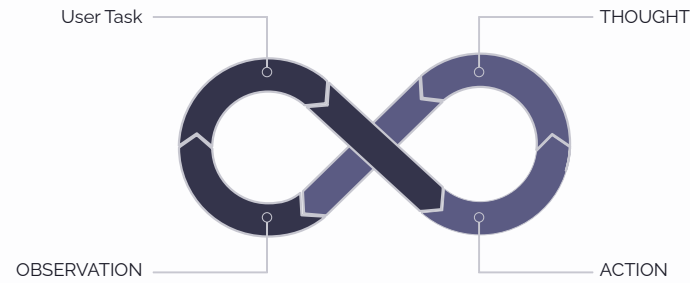
Critical for coding, math & planning tasks

Multi-Step Tasks

Enables complex reasoning across many steps

The ReAct Framework — Reasoning + Acting Together

PRINCETON & GOOGLE · 2022



ReAct =
Reasoning +
Acting

The architectural backbone of nearly every production agentic AI system today.

01

Thought

LLM writes its reasoning: *"I need to find today's weather"*

02

Action

Calls a tool: `search_web('weather Mumbai today')`

03

Observation

Reads the result and decides the next Thought

04

Final Answer

Loop ends when the task is fully resolved

Tool-Using Agents — Giving the LLM "Hands"

What is a Tool?

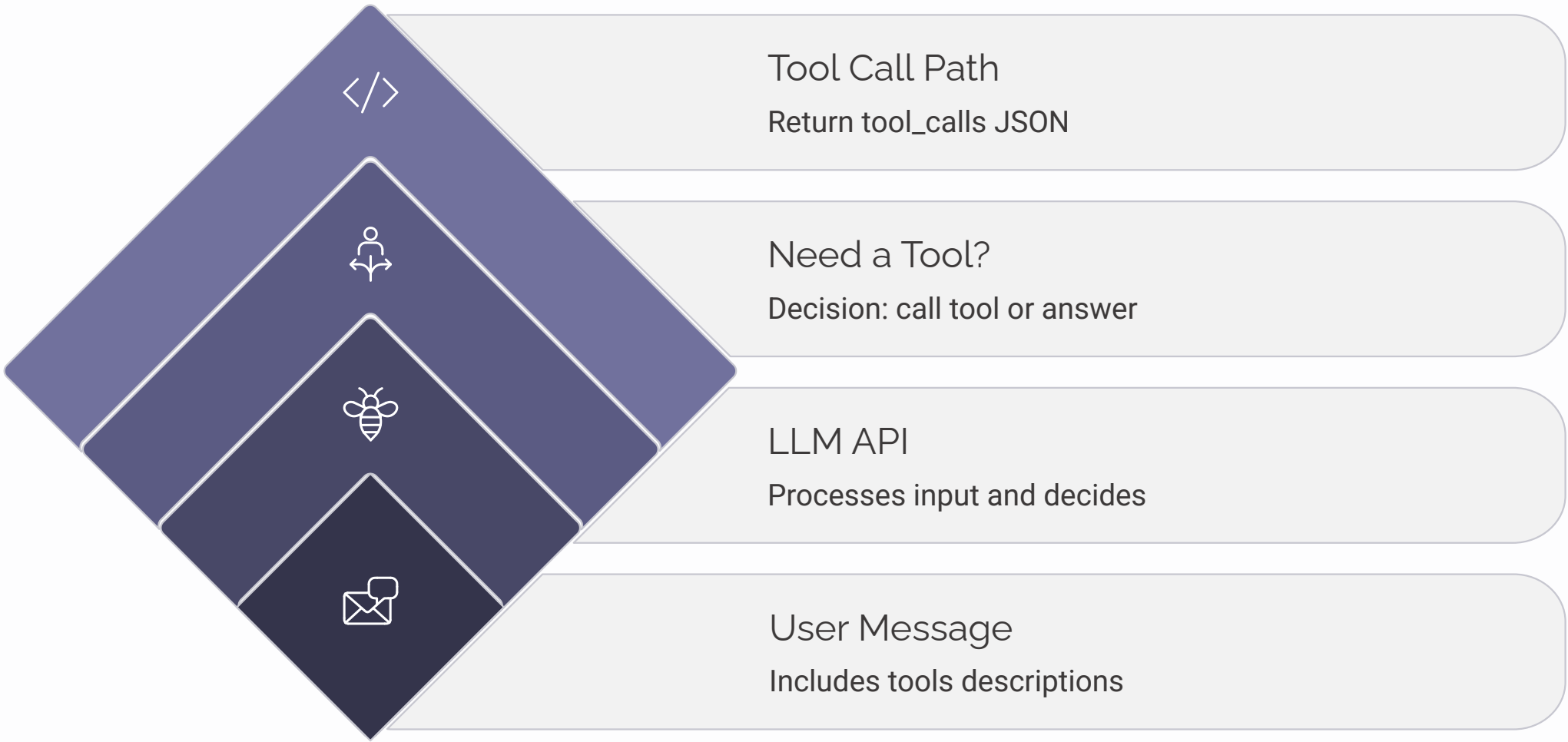
A Python function the agent can call — like a plugin that extends LLM capability beyond text. Tool names and descriptions are injected into the LLM's system prompt.

❏ **Critical Rule:** The LLM *never runs code directly* — it outputs a JSON "tool call"; your application executes it and returns the result.

Tool Type	Example	Solves Which LLM Problem?
Web Search	Google Search API	Outdated training data
Calculator	Python eval()	Arithmetic errors / hallucination
REST API	Weather API, Maps API	No live data access
Code Executor	Python sandbox	Cannot execute programs
Database	SQL query tool	No memory of user records

How Tool Calling Works — Step-by-Step Flow

 You (manager) ask an assistant to "find the cheapest hotel in Pune." The assistant calls the booking website (tool), reads the result, and reports back. The assistant never *is* the website.



- 1

Send

User message + available tools sent to LLM API
- 2

Decide

LLM returns a `tool_calls` JSON object
- 3

Execute

Your app runs the actual function
- 4

Return

Tool output added to conversation and sent back
- 5

Answer

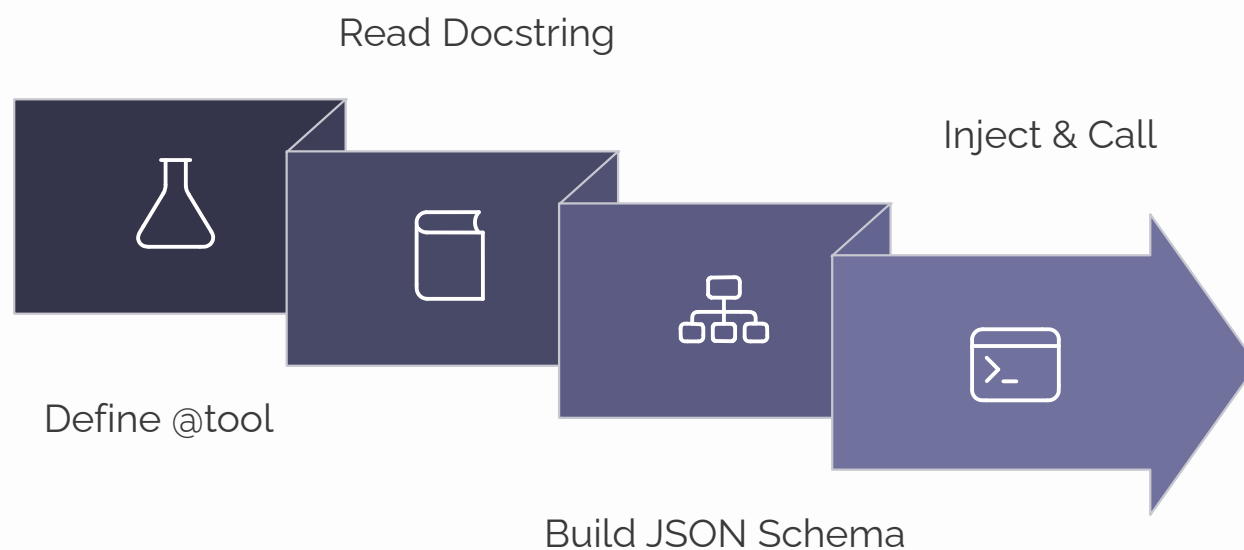
LLM uses real result to write the final answer

Practical: Defining a Tool in Python (LangChain)

LANGCHAIN FRAMEWORK

Tools = Python Functions + Docstrings

LangChain reads the docstring and type hints to build the schema the LLM uses to decide **when and how to call it**. The docstring is critical.



```
@tool
def web_search(query: str):
    """Search the web for current
    info."""
    ...

@tool
def calculator(expression: str):
    """Evaluate a math expression."""
    return eval(expression)

agent = create_agent(
    model="openai:gpt-4o",
    tools=[web_search, calculator]
)

agent.run(
    "What is 15% of today's Bitcoin
    price?"
)
```

Practical: Search + Calculator Agent — End-to-End Example



PUNE WEATHER EXAMPLE

Task: *"Current temperature in Pune, and 20% of that?"*

01

Thought

"I need live weather — I'll call web_search"

02

Action → Search

`web_search("current temperature Pune")` → **"32°C"**

03

Observation

Temperature = 32°C. Now calculate 20%.

04

Action → Calculate

`calculator("0.20 * 32")` → **"6.4"**

05

Final Answer

"Current temperature in Pune is **32°C**. 20% of that is **6.4°C**."

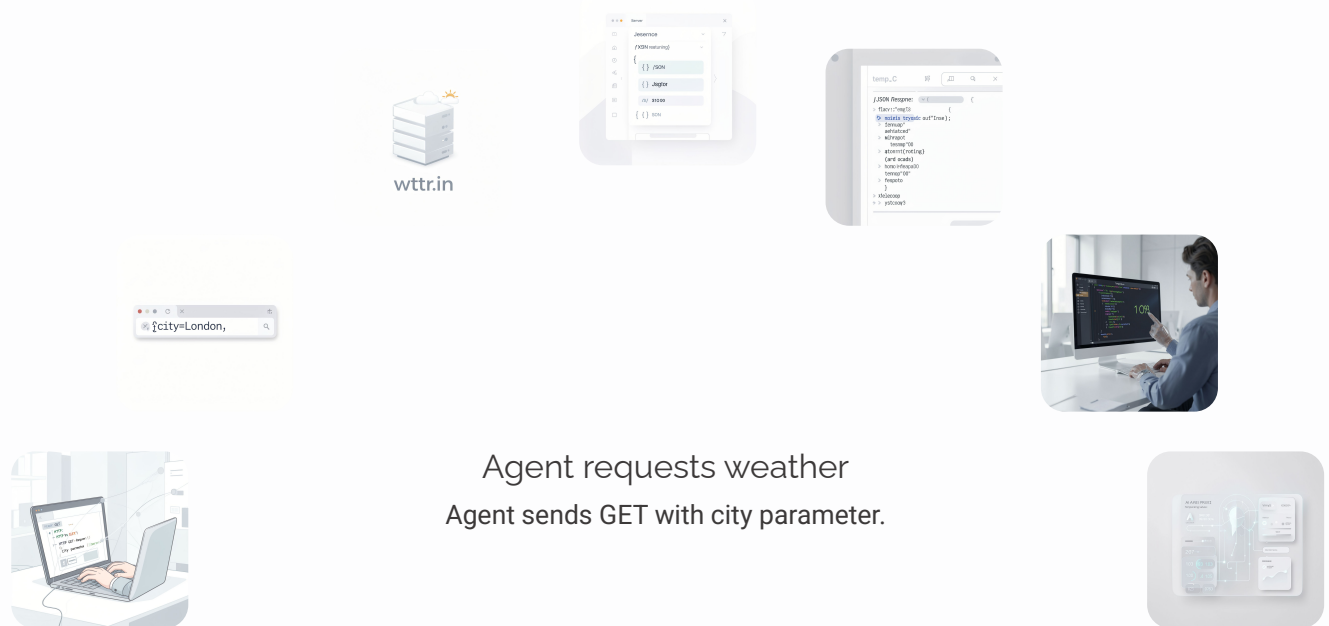
Practical: REST API Tool — Calling a Weather API

What is a REST API?

A web service you call using an HTTP request (like visiting a URL) — it returns live data in **JSON format**.

Weather API Tool in Python

```
def get_weather(city: str):  
    r = httpx.get(  
        f"https://wttr.in/{city}?format=j1"  
    )  
    return r.json()[  
        "current_condition"  
    ][0]["temp_C"]  
  
# Agent calls:  
get_weather("Pune") # returns "32"
```



Real-World REST API Tools Used by Agents

Google Maps
Directions

Razorpay
Payments

Twilio
SMS

NewsAPI
Live News

Summary — Tool-Using Agents at a Glance



Tools = Python Functions Described to LLM via docstrings – LLM picks the right one automatically	LLM Never Runs Code Outputs a JSON tool call; your app executes it and returns the result	Search Tool Solves outdated data problem with live web results
Calculator Tool Solves arithmetic errors via exact <code>eval()</code>	REST API Tool Connects agent to any real-world service	

Sources

- [What Is Agentic AI? A Plain-English Guide to AI Agents | BitsMinds](#)
- [What Is an AI Agent? How Agentic AI Actually Works in 2026 | alexi.sh](#)
- [What Is an AI Agent? A Plain-English Guide | AI/TLDR](#)
- [What Is ReAct in Agentic AI? Reasoning + Acting Framework | lowtouch.ai](#)
- [AI Agents Explained: When LLMs Start Using Tools](#)
- [Function Calling | OpenAI API](#)
- [How to Build a Tool-Using Agent with LangChain | OpenAI Cookbook](#)
- [LangChain Tools and Agents Tutorial | DevShelfHub](#)